Flujo para detectar roturas de stock

Diseño de un pipeline de datos para retail

Autor: Francisco Javier Porta Borge

Fecha: 21 de octubre de 2026

Flujo para detectar roturas de stock

Escenario asumido: cadena de retail con tiendas físicas y e-commerce. Una rotura de stock es un producto con demanda y existencias en 0 (o por debajo del mínimo de seguridad) en una ubicación concreta.

---

1. Fuente y latencia

Fuente	Qué aporta	Latencia	Por qué
Eventos de TPV / e-commerce (ventas)	Unidades vendidas por SKU y tienda	Casi tiempo real (segundos-minutos)	Es la señal más temprana de que el stock baja
WMS / ERP (inventario)	Existencias por SKU y ubicación	Micro-batch cada 5-15 min	El inventario cambia menos y no justifica streaming puro
Maestro de productos y tiendas	Stock mínimo, proveedor, categoría	Batch diario	Cambia poco
Reposiciones / pedidos a proveedor	Entradas previstas	Batch horario	Permite distinguir rotura real de reposición en camino

Decisión clave: la latencia debe responder a la pregunta *"¿cuánto cuesta enterarme tarde?"*. Para un producto de alta rotación, una hora de retraso ya es venta perdida. Para el resto, un batch cada 15 minutos basta. Streaming solo para ventas, batch para lo demás.

---

2. Capas del data lake

Raw (tal cual llega, inmutable)

- Eventos de venta originales, snapshots de inventario y ficheros del maestro.
- Particionado por fecha de ingesta, con metadatos (origen, hora de carga, lote).
- Sirve para auditar y reprocesar si cambia una regla.

Processed (limpio y estandarizado)

- Tipos corregidos, duplicados eliminados, SKU y tienda normalizados.
- Registros inválidos separados en una cuarentena (no se borran).
- Ventas e inventario unidos por `sku_id` + `store_id`, con zona horaria unificada.

Curated (listo para decidir)

- Tabla `stock_actual` por SKU-tienda: existencias, ventas últimas 24 h, cobertura en días.
- Tabla `alertas_rotura`: SKU, tienda, severidad, hora de detección, reposición prevista.
- Agregados para dashboards: roturas por categoría, tienda y proveedor.

---

3. Formato de archivo

Capa	Formato	Justificación
Ingesta de eventos (streaming)	**Avro**	Esquema explícito y evolutivo, compacto, ideal para mensajería (p. ej. Kafka)
Raw	**Se conserva el original** (JSON/CSV) y/o Avro	Fidelidad a la fuente para poder reprocesar
Processed y Curated	**Parquet**	Columnar y comprimido, rápido para consultas analíticas que leen pocas columnas
Maestros pequeños de proveedores	**CSV**	Solo como entrada, por simplicidad; no se mantiene aguas abajo

Por qué no solo uno: CSV no tiene tipos ni esquema, JSON es voluminoso y lento de consultar, Parquet no es adecuado para escribir evento a evento, y Avro es peor para analítica. Cada formato se usa donde aporta, sin añadir más de los necesarios.

---

4. Regla de calidad y KPI

Reglas de calidad (processed)

- **Validez:** `stock >= 0` y `unidades_vendidas >= 0`. Un stock negativo indica error de sincronización y se envía a cuarentena.
- **Unicidad:** un único registro por `sku_id` + `store_id` + `timestamp`.
- **Frescura:** si no llega un snapshot de inventario en más de 30 min, se levanta una alerta de calidad. Si no, una "no rotura" podría ser solo un dato viejo.
- **Integridad referencial:** todo SKU de ventas debe existir en el maestro.

KPI de negocio

- **Tasa de rotura** = SKU-tienda con stock 0 y demanda / SKU-tienda activos.
- **Tiempo de detección** = hora de alerta − hora en que el stock llegó a 0 (objetivo, por ejemplo, < 15 min).

KPI de fiabilidad del propio flujo

- **Precisión de alertas:** % de alertas confirmadas como roturas reales, para evitar que el equipo las ignore por exceso de falsos positivos.

Idea para llevarse

No todo necesita ser tiempo real ni usar la tecnología más sofisticada. Se aplica streaming donde la latencia tiene valor económico (ventas), batch donde no, y calidad antes de alertar: una alerta basada en datos dudosos es peor que no tener alerta.